



41-MAVZU · 42 TADAN · V BOB. DRF VA DEPLOY

Django loyihasini serverga joylash (Deploy)

Deploy nima, development va production farqi, hosting turlari; loyihani tayyorlash: DEBUG, SECRET_KEY, ALLOWED_HOSTS, gunicorn, whitenoise, dj-database-url, requirements.txt, collectstatic; Render'ga bosqichma-bosqich joylash, PostgreSQL, superuser; PythonAnywhere muqobili; deploy'dan keyingi tekshiruv.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- **Deploy** nima va "mening kompyuterimda ishlaydi" dan "internetda ishlaydi" ga qanday o'tishni;
- Development va production muhitlari farqini;
- Django loyihasini serverga tayyorlashni (xavfsizlik sozlamalari, statik fayllar, server dasturi);
- Loyihani **Render** bulutli platformasiga PostgreSQL bilan bosqichma-bosqich joylashni (30-loyiha);
- Joylangan saytni tekshirish va yangilashni.

Deploy nima?

Hozirgacha blogimiz faqat sizning kompyuteringizda, `127.0.0.1:8000` da ishladi — uni hech kim ko'ra olmaydi. **Deploy** (joylash) — dasturni foydalanuvchilar ishlatishi uchun internetga ulangan **serverga** o'rnatish va ishga tushirish.



O'xshatish: oshxona va restoran

Uyda ovqat pishirish (development) — o'zingiz uchun, xato qilsangiz ham hech kim bilmaydi. Restoran ochish (production) — boshqa narsa: sanitariya qoidalari (xavfsizlik), katta oshxona (server), doimiy ish vaqti (24/7), ko'p mijozlarga bir vaqtda xizmat. Retsept bir xil, lekin talablar butunlay boshqa.

	Development (ishlab chiqish)	Production (ishchi muhit)
Qayerda	Sizning kompyuteringiz	Internetdagi server
DEBUG	True — batafsil xatolar	False — xatolar yashirin (xavfsizlik!)
Server dasturi	runserver (sinov uchun)	gunicorn (ishonchli, tez)
Statik fayllar	Django o'zi beradi	collectstatic + WhiteNoise
Maxfiy kalitlar	.env fayl	Hosting'dagi muhit o'zgaruvchilari
Manzil	127.0.0.1:8000	https://maktab-blog.onrender.com

Hosting turlari (Django uchun)

Platforma	Turi	Afzalligi	E'tibor bering
Render	PaaS (bulut platformasi)	GitHub'dan avtomatik deploy, PostgreSQL, HTTPS bepul	Bepul tarifda sayt "uxlab qoladi" (birinchi ochilishi sekin); bepul baza muddati cheklangan
Railway	PaaS	Juda oson, PostgreSQL bir tugma bilan	Bepul limit cheklangan
PythonAnywhere	Python uchun maxsus hosting	O'quvchilar uchun eng oddiy, brauzerdan boshqarish	Bepul tarifda faqat SQLite/MySQL
VPS (DigitalOcean, mahalliy provayderlar)	Virtual server	To'liq nazorat	Hammasini o'zingiz sozlaysiz (Linux, Nginx)

PaaS (Platform as a Service) — server, operatsion tizim va tarmoqni platforma boshqaradi, siz faqat kodingizni berasiz. Bu darsda **Render** bilan ishlaymiz. Tariflar va bepul limitlar vaqt o'tishi bilan o'zgaradi

1-bosqich: loyihani production uchun tayyorlash

Kerakli paketlar

Terminal

```
pip install gunicorn whitenoise dj-database-url  
pip freeze > requirements.txt
```

Paket	Vazifasi
<code>gunicorn</code>	Production server dasturi (<code>runserver</code> o'rniga)
<code>whitenoise</code>	Statik fayllarni (CSS, JS) tez va xavfsiz berish
<code>dj-database-url</code>	Bazani bitta <code>DATABASE_URL</code> satridan sozlash



`gunicorn` Windows'da ishlamaydi — bu normal: u faqat serverda (Linux) ishga tushadi. Kompyuteringizda `runserver` dan foydalanishda davom eting.

settings.py — muhitga qarab sozlash

config/settings.py

```
import os
from pathlib import Path
import dj_database_url
from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")          # lokal kompyuterda .env,
serverda — muhit o'zgaruvchilari

# 🔒 Xavfsizlik
SECRET_KEY = os.environ["SECRET_KEY"]    # o'rnatilmagan bo'lsa — xato
(ataylab!)
DEBUG = os.getenv("DEBUG", "False") == "True" # standart — False
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "127.0.0.1,localhost").split(",")
CSRF_TRUSTED_ORIGINS = [f"https://{h}" for h in ALLOWED_HOSTS if h not in
("127.0.0.1", "localhost")]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",      # ←
    SecurityMiddleware'dan keyin
    "django.contrib.sessions.middleware.SessionMiddleware",
    # ... qolganlari o'zgarishsiz
]

# 🐘 Baza: serverda DATABASE_URL, lokal kompyuterda .env'dagi qiymatlar
DATABASES = {
    "default": dj_database_url.config(
        default=f"postgres://{os.getenv('DB_USER')}:
{os.getenv('DB_PASSWORD')}@"
        f"{os.getenv('DB_HOST', 'localhost')}:{os.getenv('DB_PORT',
'5432')}/{os.getenv('DB_NAME')}",
        conn_max_age=600,
    )
}

# 📁 Statik fayllar
STATIC_URL = "static/"
```

```

STATIC_ROOT = BASE_DIR / "staticfiles" # collectstatic shu
yerga yig'adi
STORAGES = {
    "default": {"BACKEND": "django.core.files.storage.FileSystemStorage"},
    "staticfiles": {"BACKEND":
"whitenoise.storage.CompressedManifestStaticFilesStorage"},
}

# 🛡️ Production'da qo'shimcha himoya
if not DEBUG:
    SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
    SESSION_COOKIE_SECURE = True
    CSRF_COOKIE_SECURE = True

```

❌ Production'ning oltin qoidalari

1. `DEBUG = False` — aks holda xato sahifasi buzg'unchiga kodingiz, sozlamalar va hatto parollar haqida ma'lumot beradi.
2. `SECRET_KEY` **maxfiy va yangi** — GitHub'dagi yoki darsdagi kalitni ishlatmang. Yangisini yarating: `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`
3. `ALLOWED_HOSTS` — faqat sizning domeningiz.

Lokal kompyuteringizdagi `.env` ga `DEBUG=True` qo'shib qo'ying — shunda kompyuteringizda ishlash qulay qoladi.

Build skripti

Server har deploy'da bajaradigan buyruqlar:

build.sh (loyiha ildizida)

```

#!/usr/bin/env bash
set -o errexit # biror buyruq xato bersa - to'xtash

pip install -r requirements.txt
python manage.py collectstatic --no-input
python manage.py migrate

```

Terminal

```
git update-index --chmod=+x build.sh    # Windows'da faylni "bajariladigan"  
qilish (Git orqali)  
# macOS/Linux: chmod +x build.sh
```

Lokal tekshiruv

Terminal

```
python manage.py check --deploy          # Django production uchun maslahatlar  
beradi  
python manage.py collectstatic          # staticfiles/ papkasi paydo bo'ladi  
(.gitignore ga qo'shing)
```

.gitignore

```
venv/  
.env  
__pycache__/  
staticfiles/  
media/
```

GitHub'ga push

Terminal

```
git add .  
git commit -m "Production uchun tayyorlandi: gunicorn, whitenoise, dj-database-  
url"  
git push
```

2-bosqich: Render'ga joylash

1 render.com da **GitHub** akkauntingiz bilan ro'yxatdan o'ting.

2 **PostgreSQL bazasi yarating:** Dashboard → **New +** → **Postgres** → nom (`maktab-blog-db`), region (Frankfurt — O'zbekistonga yaqinroq), bepul tarif → **Create Database**. Tayyor bo'lgach, **Internal Database URL** ni nusxalang (`postgresql:// ...`).

3 **Web Service yarating:** **New +** → **Web Service** → GitHub repozitoriyangizni tanlang.

Maydon	Qiymat
Name	<code>maktab-blog</code> (manzil: <code>maktab-blog.onrender.com</code>)
Language / Runtime	Python 3
Build Command	<code>./build.sh</code>
Start Command	<code>gunicorn config.wsgi:application</code>
Instance Type	Free

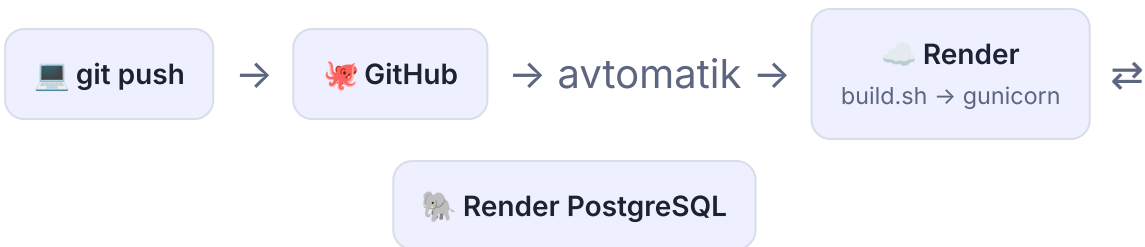
4 **Environment Variables** (muhit o'zgaruvchilari) bo'limida qo'shing:

Key	Value
<code>SECRET_KEY</code>	yangi yaratilgan uzun tasodifiy kalit
<code>DEBUG</code>	<code>False</code>
<code>ALLOWED_HOSTS</code>	<code>maktab-blog.onrender.com</code>
<code>DATABASE_URL</code>	2-qadamdagi Internal Database URL
<code>PYTHON_VERSION</code>	kompyuteringizdagi versiya, masalan <code>3.12.7</code>

5 **Create Web Service** bosing. **Logs** oynasida jarayonni kuzating: paketlar o'rnatiladi → collectstatic → migrate → gunicorn ishga tushadi. "**Your service is live** 🎉" yozuvini kuting (bir necha daqiqa).

6 **Admin yaratish:** servis sahifasida **Shell** yorlig'i (agar tarifingizda mavjud bo'lsa) → `python manage.py createsuperuser` . Shell mavjud bo'lmasa, lokal kompyuterdan **External Database URL** bilan bajarish mumkin: `.env` dagi baza sozlamalari o'rniga vaqtincha `DATABASE_URL= ... external ...` qo'yib, `python manage.py createsuperuser` (keyin qaytaring!).

7 Saytingizni oching: `https://maktab-blog.onrender.com` 🎉 Havolani do'stlaringiz va ota-onangizga yuboring!



Har bir `git push` dan keyin Render saytni avtomatik qayta joylaydi (CI/CD)

3-bosqich: deploy'dan keyin

Tekshiruv ro'yxati

Tekshiruv	✓
Bosh sahifa ochiladi, CSS ishlayapti (dizayn buzilmagan)	☐
<code>/admin/</code> ga kirish mumkin, maqola qo'shish ishlaydi	☐
Ro'yxatdan o'tish, kirish, maqola yaratish/tahrirlash/o'chirish ishlaydi	☐
<code>/maqola/9999/</code> — chiroyli 404 sahifa (sariq Django xatosi EMAS!)	☐
<code>/api/maqolalar/</code> JSON qaytaradi	☐
Manzilda  (HTTPS)	☐
Telefonda to'g'ri ko'rinadi	☐
40-mavzudagi JS front-end'ni Netlify'ga joylab, <code>API</code> manzilini Render'dagisiga almashtiring va <code>CORS_ALLOWED_ORIGINS</code> ga Netlify manzilini qo'shing	☐

Yangilash

Terminal

```
# Kodni o'zgartirdingiz → lokal sinadingiz → yubordingiz
git add .
git commit -m "Izohlar funksiyasi qo'shildi"
git push          # Render avtomatik qayta joylaydi; yangi migratsiyalar
build.sh'da bajariladi
```

Xatolarni topish

- **Logs** — barcha xatolar shu yerda. "Build failed" bo'lsa — build log'ini, sayt 500 qaytarsa — runtime log'ini o'qing.
- `DisallowedHost` → `ALLOWED_HOSTS` da domen yo'q yoki xato yozilgan.
- CSS yo'q (oddiy matn ko'rinishidagi sayt) → WhiteNoise middleware yoki `collectstatic` muammosi.
- `CSRF verification failed` (forma yuborishda) → `CSRF_TRUSTED_ORIGINS` da `https://` bilan domen yo'q.
- `ModuleNotFoundError` → paket `requirements.txt` ga yozilmagan (`pip freeze > requirements.txt` ni unutgansiz).
- Hech qachon muammoni "tuzatish" uchun serverda `DEBUG=True` qo'ymang!



Media fayllar haqida

Bulutli platformalarning disk xotirasi ko'pincha **vaqtinchalik**: har deploy'da foydalanuvchilar yuklagan rasmlar o'chib ketishi mumkin. Haqiqiy loyihalarda media fayllar alohida xizmatda (Cloudinary, Amazon S3) saqlanadi. O'quv loyihasi uchun bu muammo emas, lekin buni bilib qo'ying.

Muqobil: PythonAnywhere (qisqacha)

- 1 pythonanywhere.com da bepul "Beginner" akkaunt oching.
- 2 **Consoles → Bash:** `git clone https://github.com/USERNAME/maktab_blog.git`, virtual muhit yarating va `pip install -r requirements.txt`.
- 3 **Web → Add a new web app → Manual configuration** → Python versiyasi. WSGI fayliga loyiha yo'li va `DJANGO_SETTINGS_MODULE = "config.settings"` ni yozing.

- 4 Bepul tarifda PostgreSQL yo'q — `DATABASE_URL` o'rniga SQLite (`sqlite:///db.sqlite3`) ishlatiladi. Static fayllar uchun **Web** sahifasida `/static/` → `staticfiles` papkasini ko'rsating.
- 5 **Reload** — sayt `USERNAME.pythonanywhere.com` da.

Amaliy mashg'ulot (30-loyiha)

- 1 Blog loyihangizni production uchun tayyorlang va `python manage.py check --deploy` ogohlantirishlarini o'qing.
- 2 Render'da PostgreSQL va Web Service yarating, loyihani joylang.
- 3 Tekshiruv ro'yxatidagi barcha bandlarni bajaring.
- 4 Kichik o'zgarish kiriting (masalan, footer matni), push qiling va avtomatik qayta joylashni kuzating.
- 5 README.md ga sayt havolasini, skrinshotlarni va "Qanday ishga tushirish" bo'limini qo'shing.

O'zingizni tekshiring

? **Production serverda** `DEBUG` qanday bo'lishi kerak va nima uchun?

- A. True — xatolarni ko'rish uchun
- B. False — xato sahifalari maxfiy ma'lumotlarni oshkor qilmasligi uchun
- C. Ahamiyati yo'q
- D. None

? **Production'da `runserver` o'rniga ishlatiladigan server dasturi?**

- A. whitenoise
- B. pip
- C. psqI
- D. gunicorn

? **Barcha statik fayllarni bitta papkaga yig'uvchi buyruq?**

- A. `python manage.py collectstatic`
- B. `python manage.py migrate`
- C. `python manage.py static`
- D. `pip install static`

? **Sayingiz `DisallowedHost` xatosini berdi. Sababi?**

- A. Baza ulanmagan
- B. CSS topilmadi
- C. Domen `ALLOWED_HOSTS` da yo'q
- D. Parol noto'g'ri

? **Maxfiy kalitlar (`SECRET_KEY`, baza paroli) production'da qayerda saqlanadi?**

- A. settings.py ichida
- B. Hosting'ning muhit o'zgaruvchilarida (environment variables)
- C. GitHub README'da
- D. HTML shablonda

? Render'ga ulangan loyihani yangilash uchun nima qilinadi?

- A. Serverga qayta ro'yxatdan o'tish
- B. Loyihani o'chirib, qayta yaratish
- C. O'zgarishlarni GitHub'ga `git push` qilish
- D. Hech narsa — yangilab bo'lmaydi



Uyga vazifa

1. IV bob uyga vazifasidagi loyihangizni (kutubxona / talabalar / e'lonlar) ham internetga joylang.
2. Barcha joylangan loyihalarangiz havolalarini GitHub profilingizdagi README'ga va portfolio saytingizga qo'shing — bu sizning birinchi "rezyume"ingiz!



Yodda tuting

- Deploy — loyihani internetdagi serverga joylash; production ≠ development.
- Tayyorlash: `DEBUG=False`, maxfiy `SECRET_KEY`, `ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS`; `gunicorn`, `whitenoise`, `django-database-url`; `requirements.txt`.
- `build.sh`: `install` → `collectstatic` → `migrate`; start: `gunicorn config.wsgi:application`.
- Render: Postgres + Web Service + Environment Variables → har `git push` da avtomatik deploy.
- Muammolarni Logs orqali toping; serverda hech qachon `DEBUG=True` qo'ymang.

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. Production serverda DEBUG qanday bo'lishi kerak va nima uchun? — **B) False — xato sahifalari maxfiy ma'lumotlarni oshkor qilmasligi uchun**
2. Production'da runserver o'rniga ishlatiladigan server dasturi? — **D) gunicorn**
3. Barcha statik fayllarni bitta papkaga yig'uvchi buyruq? — **A) python manage.py collectstatic**
4. Saytingiz DisallowedHost xatosini berdi. Sababi? — **C) Domen ALLOWED_HOSTS da yo'q**
5. Maxfiy kalitlar (SECRET_KEY, baza paroli) production'da qayerda saqlanadi? — **B) Hosting'ning muhit o'zgaruvchilarida (environment variables)**
6. Render'ga ulangan loyihani yangilash uchun nima qilinadi? — **C) O'zgarishlarni GitHub'ga git push qilish**